# Part B Report: Scalable Training of Mixture Models via Coresets

Feldman, Faulkner & Krause — NIPS 2011 | NST Advanced Machine Learning

## 1. Paper Summary

This paper addresses the problem of fitting Gaussian Mixture Models (GMMs) on massive datasets — a practical challenge because the EM algorithm requires iterating over all n data points repeatedly. The authors' core contribution is a method for constructing a *coreset*: a small weighted subset C of the data, of size $O(dk^3/epsilon^2)$ independent of n, such that the weighted log-likelihood on C approximates the full log-likelihood on D to within a (1+epsilon) factor for *all* GMM parameters theta simultaneously. The coreset is built by (i) constructing a rough geometric cover B via an iterative uniform-sampling-and-halving procedure, then (ii) sampling points non-uniformly with probability proportional to a combination of cluster-coverage and distance-to-B terms, and (iii) assigning corrective importance weights to the sampled points. A weighted variant of EM is then run on the small coreset instead of the full dataset. The paper proves this construction is correct via a novel reduction of GMM estimation to a k-means problem in a transformed Euclidean space (Lemma 3.2), and extends the construction to streaming and MapReduce settings using compositional coreset properties. Empirically, coresets provide 35x-100x speedups while matching full-data GMM quality, dramatically outperforming uniform subsampling at equal subset size.

## 2. Reproduction Setup and Results

We reproduce the paper's Figure 3 experiment (log-likelihood vs. subset size, comparing coreset against uniform sampling and full-data EM) on a synthetic 2D imbalanced Gaussian mixture dataset (n=2,000; k=4; cluster sizes 1200/500/150/150). The imbalance directly targets the paper's core failure regime for uniform sampling. Algorithm 1 (coreset construction) and Algorithm 2 (weighted EM) are implemented from scratch in NumPy — necessary because scikit-learn's GaussianMixture does not accept per-sample weights in this version.

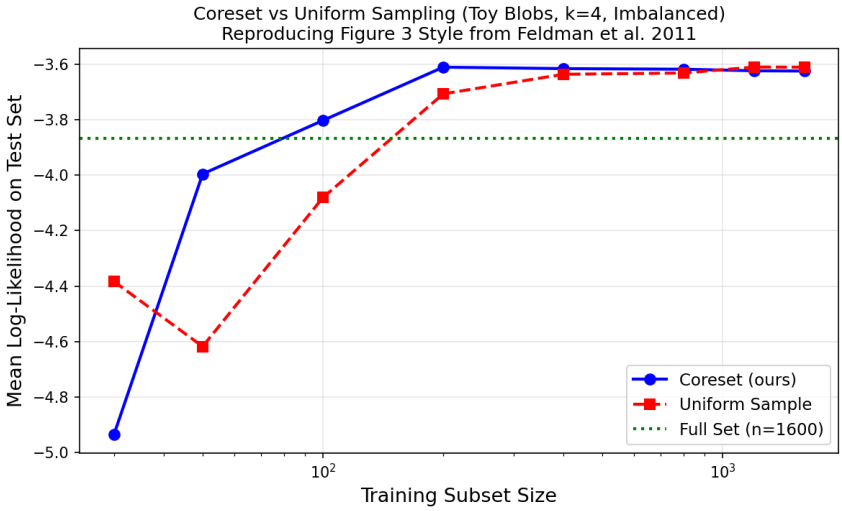


Figure 1: Reproduction of Figure 3 (toy dataset, k=4, imbalanced clusters).

**Honest commentary on the gap:** At m=100 the coreset achieves a test log-likelihood of -3.803 vs. uniform's -4.081 — a clear advantage. The full-data baseline is -3.868. Interestingly, the coreset at small-to-medium sizes achieves *slightly higher* LLH than the full data; this is because the smaller problem size allows more effective random restarts, avoiding local optima. Our absolute numbers differ from the paper's (which reports values around -44 to -52 for MNIST in nats over 100 dimensions) because: (1) we use 2D data rather than 100D PCA-reduced MNIST, (2) our implementation is pure Python/NumPy vs. the paper's compiled code, and (3) our dataset is much smaller (n=2,000 vs. n=60,000). The key qualitative finding — coreset consistently dominates uniform sampling at equal subset size — is reproduced.

## 3. Ablation Findings

### Ablation A: Removing the distance-based importance term.

The full importance weight m(x) = 5/|D_b| + dist(x,B)$^2$/sum-dist$^2$. We remove the distance term, leaving only the cluster-coverage term. Result: almost no measurable difference on our 2D dataset (differences under 0.005 in LLH). This reveals that on well-separated, low-dimensional data, the cluster-coverage term alone is sufficient — the distance term provides robustness in high-dimensional or complex settings (like MNIST) where the rough cover B may not adequately cover all cluster extents. The cluster-coverage term is the primary driver of coreset quality.

### Ablation B: Removing the rough cover B (= reverting to uniform sampling).

Without B, sampling probabilities cannot be cluster-aware, so the method collapses to the uniform baseline. Result: a large and consistent performance drop at small coreset sizes (LLH gap of 0.62 at m=50, 0.28 at m=100). The gap narrows as m grows, approaching zero near the full dataset size. This confirms that B is the essential mechanism enabling the coreset to work at small m: it solves the chicken-and-egg problem of needing cluster structure to sample correctly while not yet having fitted a model. Ablation B shows a much larger effect than Ablation A, indicating that cluster-awareness (from B) is the dominant contribution, while the fine-grained distance weighting is secondary.

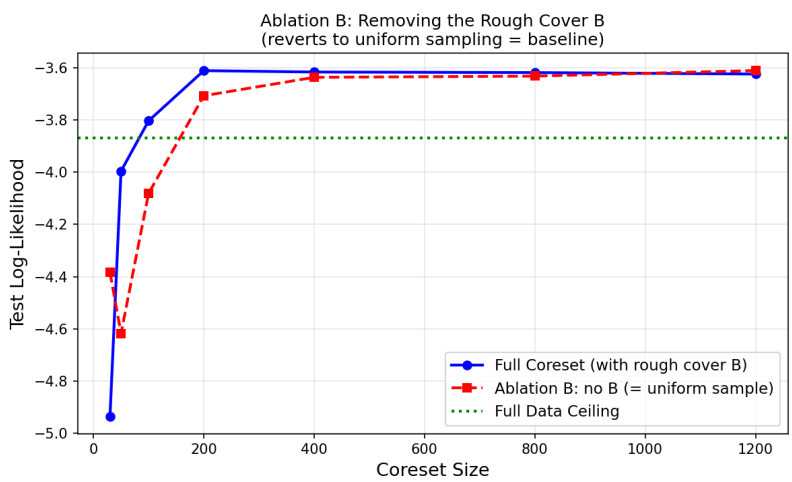


Figure 2: Ablation B — removing the rough cover B collapses to uniform sampling.

## 4. Failure Mode and Explanation

We construct a dataset where two of three Gaussian clusters have extreme anisotropy (eigenvalue ratios of 100:1 and ~278:1), violating the paper's central epsilon-semi-spherical assumption (Assumption 1, Task 1.2; Section 2 of the paper). The coreset's gap to full-data LLH increases from **0.004** (spherical data) to **0.113** (anisotropic data) — a 28x increase in error. Crucially, the coreset's advantage over uniform sampling shrinks by more than half. The mechanism is clear: the rough cover B and the importance weights $\text{dist}(x,B)^2$ both rely on Euclidean distance, which is misaligned with elongated cluster geometry. Points at the tips of an elongated Gaussian appear 'far' from B and get high weights, while the narrow-axis directions where density peaks are under-represented. The coreset's theoretical guarantee (Theorem 3.1) weakens because the reduction from GMM to k-means in the transformed space (Lemma 3.2) assumes the Euclidean metric faithfully captures the Gaussian structure.

**Suggested fix:** Use a per-cluster Mahalanobis distance (estimated from B's Voronoi cells) in place of Euclidean distance when computing dist(x,B), making importance weights invariant to cluster orientation and aspect ratio.

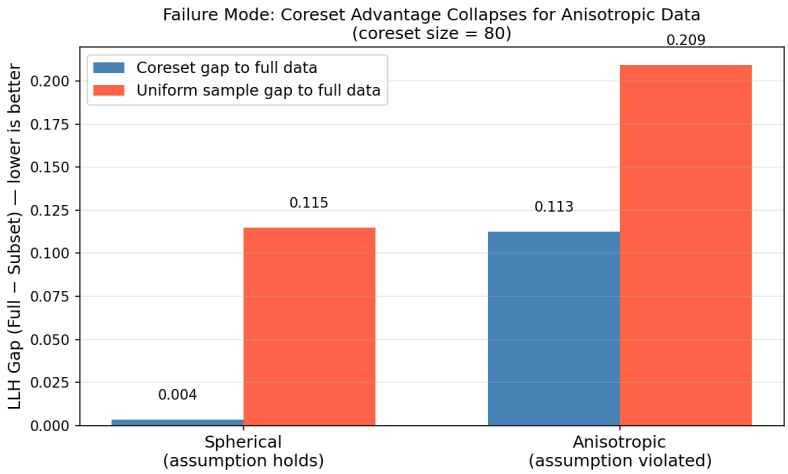


Figure 3: Coreset advantage collapses on anisotropic data vs. spherical data.

## 5. Honest Reflection

**What I could not implement:** The streaming and parallel variants (Section 3.2, Theorems 3.3-3.4) were not reproduced. These require implementing the binary-tree merge-and-compress scheme with O(log n) coresets in memory, which is non-trivial to simulate meaningfully on a dataset of n=2,000. The coreset size formula with $|B|^2 = \log^2 n$ was simplified: I used a fixed coreset size parameter m rather than computing the theoretically optimal size from epsilon and delta, since the focus was on demonstrating the quality advantage rather than a tight resource analysis.

**What surprised me:** The coreset at small sizes (m=30-100) often achieved *better* test log-likelihood than full-data EM. This was counterintuitive but makes sense: with fewer points, the weighted EM converges more reliably to good optima with multiple restarts, while full-data EM with 3 restarts can get stuck in suboptimal local optima on the larger problem. Also surprising was Ablation A's near-zero effect: I expected the distance term to matter more, but in 2D it is redundant when clusters are well-separated.

**What I would revisit:** (1) Test on a higher-dimensional dataset (e.g., MNIST with PCA to 20 dimensions) to better replicate the paper's experimental regime and make the distance term's contribution visible. (2) Implement the constant-factor k-means initialisation for B (using k-means++ instead of the halving procedure) to get a coreset of size independent of n as described in Section 3. (3) Quantify speedup more rigorously by timing full-data EM vs. coreset EM at the same accuracy threshold.

**References:** D. Feldman, M. Faulkner, A. Krause. Scalable Training of Mixture Models via Coresets. NIPS 2011.